CSCI 423
Introduction to Algorithms

DR. RAAFAT ELFOULY

Jeffrey J. McConnell

Analysis of Algorithms

Second Edition

An Active Learning Approach

Chapter 5
Numeric Algorithms

Chapter Outline

- Calculating polynomials
- Matrix multiplication
- Linear equations

Prerequisites

- Before beginning this chapter, you should be able to:
  - Do simple algebra
  - Evaluate polynomials
  - Describe growth rates and order

Goals

- At the end of this chapter you should be able to:
  - Evaluate a polynomial by Horner's method
  - Explain matrix multiplication
  - Explain the Gauss-Jordan method for solving systems of linear equations

Numeric Algorithm Efficiency

- Numeric programs will do a particular calculation a large number of times

- Small improvements can result in significant time savings because of how many times the calculation is done

Numeric Analysis Basics

- We count additions and multiplications
- Because additions are typically much faster than multiplications, reducing the multiplications by increasing the additions can still result in an improvement
- The analysis will be based on the highest power in a polynomial or the size of the matrices we are working with

Calculating Polynomials

- We will use a generic polynomial form of:

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

  where the coefficient values are known constants
- The value of *x* will be the input and the result is the value of the polynomial using this *x* value

- Develop algorithm
- Work in a group of two

Standard Evaluation Algorithm

```
result = a[0] + a[1]*x
xPower = x
for i = 2 to n do
   xPower = xPower * x
   result = result + a[i]*xPower
end for
return result
```

Do the Analysis
how many operations we have??

- Before the loop, there is
- The `for` loop
- The Total

Analysis

- Before the loop, there is
  - One multiplication
  - One addition
- The `for` loop is done *N*-1 times
  - There are two multiplications in the loop
  - There is one addition in the loop
- There are a total of
  - 2*N* – 1 multiplications
  - *N* additions

Horner's Method

- Based on the factorization of a polynomial
- The generic equation is factored as:

$$p(x) = (\{\cdots[(a_n x + a_{n-1}) * x + a_{n-2}] * x + \cdots + a_2\} * x + a_1) * x + a_0$$

- For example, the equation:

$$p(x) = x^3 - 5x^2 + 7x - 4$$

- would be factored as:

$$p(x) = [(x - 5) * x + 7] * x - 4$$

- Develop algorithm
- Work in a group of two

Horner's Method Algorithm

```
result = a[n]
for i = n - 1 down to 0 do
   result = result * x
   result = result + a[i]
end for
return result
```

Do the Analysis

- The `for` loop??

- The Total?

Analysis

- The `for` loop is done *N* times
  - There is one multiplication in the loop
  - There is one addition in the loop
- There are a total of
  - *N* multiplications
  - *N* additions
- Saves *N* – 1 multiplications over the standard algorithm

Polynomial Algorithm Comparison

- For the example polynomial:
  - Standard algorithm:
    - 5 multiplications and 3 additions
  - Horner's method
    - 3 multiplications and 3 additions

$$p(x) = x^3 - 5x^2 + 7x - 4$$

Polynomial Algorithm Comparison

- In general, for a polynomial of degree *N*:
  - Standard algorithm:
    - $2N - 1$ multiplications and N additions
  - Horner's method
    - N multiplications and N additions

Matrix Multiplication

- Two matrices can be multiplied if the number of columns in the first is the same as the number of rows in the second
- Each row of the first matrix is multiplied by each column of the second matrix
- The value at location *i*, *j* of the result is from the addition of the products when row *i* of the first matrix is multiplied by row *j* of the second matrix

Matrix Multiplication

- An example of this process is:

$$\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = \begin{bmatrix} a1+b2+c3 & a4+b5+c6 \\ d1+e2+f3 & d4+e5+f6 \end{bmatrix}$$

- Develop the algorithm

Matrix Multiplication Algorithm

```
for i = 1 to a do
   for j = 1 to c do
      R[i,j] = G[i,1] * H[1,j]
      for k = 2 to b
         R[i,j] = R[i,j] + G[i,k] * H[k,j]
      end for k
   end for j
end for i
```

- What is its complexity ???
- How many additions and multiplications?

Analysis

- The "for *k*" loop runs *b* – 1 times and does one multiplication and addition each time
- The "for *j*" loop runs *c* times and does *b* multiplications and *b* – 1 additions each time
- The "for *i*" loop runs a times and does *b***c* multiplications and (*b* – 1)**c* additions each time
- Therefore, the algorithm does *a***b***c* multiplications and *a**(*b* – 1)**c* additions

Strassen's Matrix Mutiplication

- Uses a set of seven formulas to multiply two 2 X 2 matrices
- Does not rely on elements being commutative under multiplication
- It can be applied recursively, in other words, two 4 X 4 matrices can be multiplied by treating each as a 2 X 2 matrix of 2 X 2 matrices

Strassen's Algorithm (1969)

Step 1 : recursively compute only 7 (cleverly chosen) products

Step 2 : do the necessary (clever) additions + subtractions (still $\theta(n^2)$ time)

Fact : better than cubic time!

Strassen's Formulas

- First we calculate a set of temporary values:

$x_1 = (G[1,1] + G[2,2]) * (H[1,1] + H[2,2])$

$x_2 = (G[2,1] + G[2,2]) * H[1,1]$

$x_3 = G[1,1] * (H[1,2] - H[2,2])$

$x_4 = G[2,2] * (H[2,1] - H[1,1])$

$x_5 = (G[1,1] + G[1,2]) * H[2,2]$

$x_6 = (G[2,1] - G[1,1]) * (H[1,1] + H[1,2])$

$x_7 = (G[1,2] - G[2,2]) * (H[2,1] + H[2,2])$

Strassen's Formulas

- The result is then calculated by:

$$R[1,1] = x_1 + x_4 - x_5 + x_7$$

$$R[2,1] = x_2 + x_4$$

$$R[1,2] = x_3 + x_5$$

$$R[2,2] = x_1 + x_3 - x_2 + x_6$$

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \qquad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

$$P_1 = A * (F - H)$$
$$P_2 = H * (A + B)$$
$$P_3 = E * (C + D)$$
$$P_4 = D * (G - E)$$
$$P_5 = (A + D) * (E + H)$$
$$P_6 = (B - D) * (G + H)$$
$$P_7 = (A - C) * (E + F)$$

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \qquad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

$$P_1 = A(F - H)$$
$$P_2 = H(A + B)$$
$$P_3 = E(C + D)$$
$$P_4 = D(G - E)$$
$$P_5 = (A + D) * (E + H)$$
$$P_6 = (B - D) * (G + H)$$
$$P_7 = (A - C) * (E + F)$$

$$XY = \begin{bmatrix} P_6 + P_5 + P_4 - P_2 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

- Calculate XY by claculatin P1 to P7
- Substitute for them in XY

Analysis

- These formulas require 7 multiplications and 18 additions to multiply two 2 X 2 matrices
- The real savings occur when this is applied recursively and we do approximately $N^{2.81}$ multiplications and $6N^{2.81} – 6N^2$ additions
- Though not used in practice, Strassen's method is important because it was the first algorithm that is faster than $O(N^3)$

Linear Equations

- A system of linear equations is a set of *N* equations in *N* unknowns:

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \ldots + a_{1N}x_N = b_1$$
$$a_{21}x_1 + a_{22}x_2 + a_{a3}x_3 + \ldots + a_{2N}x_N = b_2$$
$$\vdots$$
$$a_Nx_1 + a_{N2}x_2 + a_{N3}x_3 + \ldots + a_{NN}x_N = b_N$$

Linear Equations

- When these equations are used, the coefficients (*a* values) are constants and the results (*b* values) are typically input values

- We want to determine the *x* values that satisfy these equations and produce the indicated results

Linear Equation Example

- An example set of linear equations with three unknowns is:

$$2x_1 - 4x_2 + 6x_3 = 14$$
$$6x_1 - 6x_2 + 6x_3 = 24$$
$$4x_1 + 2x_2 + 2x_3 = 18$$

Solving Linear Equations

- One method to determine a solution would be to substitute one equation into another

- For example, we solve the first equation for x_1 and then substitute this into the rest of the equations

- This substitution reduces the number of unknowns and equations

Solving Linear Equations

- If we repeat this, we get to one unknown and one equation and can then back up to get the values of the rest

- If there are many equations, this process can be time consuming, prone to error, and is not easily computerized

Gauss-Jordan Method

- This method is based on the previous idea

- We store the equation constants in a matrix with *N* rows and *N*+1 columns
- For the example, we would get:

$$\begin{bmatrix} 2 & -4 & 6 & 14 \\ 6 & -6 & 6 & 24 \\ 4 & 2 & 2 & 18 \end{bmatrix}$$

Gauss-Jordan Method

- We perform operations on the rows until we eventually have the identity matrix in the first *N* columns and then the unknown values will be in the final column:

$$\begin{bmatrix} 1 & 0 & 0 & x_1 \\ 0 & 1 & 0 & x_2 \\ 0 & 0 & 1 & x_3 \end{bmatrix}$$

Gauss-Jordan Method

- On each pass, we pick a new row and divide it by the first element that is not zero
- We then subtract multiples of this row from all of the others to create all zeros in a column except in this row
- When we have done this *N* times, each row will have one value of 1 and the last column will have the unknown values

Example

- Consider the example again:

$$\begin{bmatrix} 2 & -4 & 6 & 14 \\ 6 & -6 & 6 & 24 \\ 4 & 2 & 2 & 18 \end{bmatrix}$$

- We begin by dividing the first row by 2, and then subtract 6 times it from the second row and 4 times it from the third row

- Solve the example

$$\begin{bmatrix} 2 & -4 & 6 & 14 \\ 6 & -6 & 6 & 24 \\ 4 & 2 & 2 & 18 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -2 & 3 & 7 \\ 0 & 6 & -12 & -18 \\ 0 & 10 & -10 & -10 \end{bmatrix}$$

Example

$$\begin{bmatrix} 1 & -2 & 3 & 7 \\ 0 & 6 & -12 & -18 \\ 0 & 10 & -10 & -10 \end{bmatrix}$$

- Now, we divide the second row by 6, and then subtract -2 times it from the first row and 10 times it from the third row

Example

$$\begin{bmatrix} 1 & 0 & -1 & 1 \\ 0 & 1 & -2 & -3 \\ 0 & 0 & 10 & 20 \end{bmatrix}$$

- Now, we divide the third row by 10, and then subtract -1 times it from the first row and -2 times it from the second row

Example

- This gives the result of:

$$\begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}$$

- And we have that $x_1 = 3$, $x_2 = 1$, and $x_3 = 2$

Concerns

- In practice, computer round-off error give inaccurate results
- If the matrix is singular, one row will be just a multiple of another row and we will get a divide by zero error
- There are other algorithms that can also handle these problems but they are more appropriate for a numerical analysis course

Jeffrey J. McConnell

Analysis of Algorithms

Second Edition

An Active Learning Approach

Chapter 8, Part I

Graph Algorithms

Chapter Outline

- Graph background and terminology
- Data structures for graphs
- Graph traversal algorithms
- Minimum spanning tree algorithms
- Shortest-path algorithm
- Biconnected components
- Partitioning sets

Prerequisites

- Before beginning this chapter, you should be able to:
  - Describe a set and set membership
  - Use two-dimensional arrays
  - Use stack and queue data structures
  - Use linked lists
  - Describe growth rates and order

Goals

- At the end of this chapter you should be able to:
  - Describe and define graph terms and concepts
  - Create data structures for graphs
  - Do breadth-first and depth-first traversals and searches
  - Find the minimum spanning tree for a connected graph
  - Find the shortest path between two nodes of a connected graph
  - Find the biconnected components of a connected graph

- What is a graph?

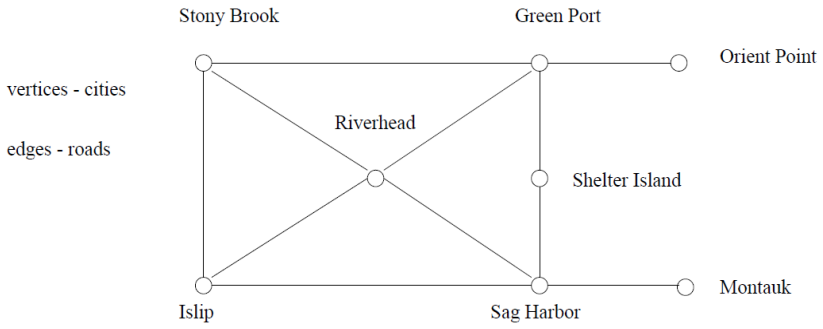
Graphs

Graphs are one of the unifying themes of computer science. A graph $G = (V, E)$ is defined by a set of *vertices* V, and a set of *edges* E consisting of ordered or unordered pairs of vertices from V.

- Why we need them?
- Examples?

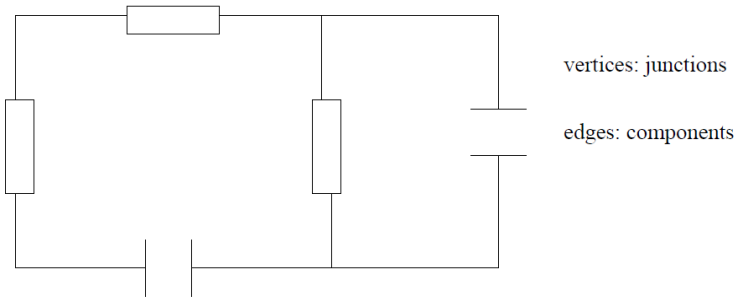
Road Networks

In modeling a road network, the vertices may represent the cities or junctions, certain pairs of which are connected by roads/edges.



Electronic Circuits

In an electronic circuit, with junctions as vertices as components as edges.



Graph Types

- A directed graph edges' allow travel in one direction
- An undirected graph edges' allow travel in either direction

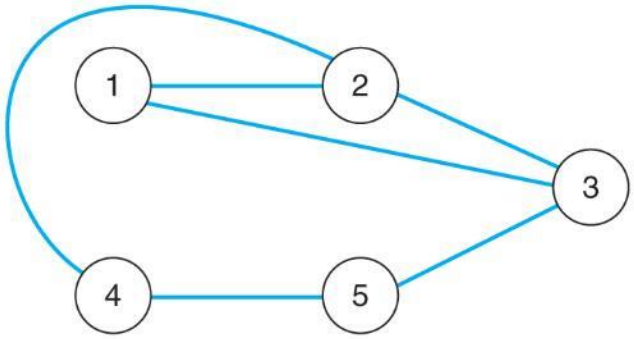


FIGURE 8.1A
The graph G = ({1, 2, 3, 4, 5}, {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}})

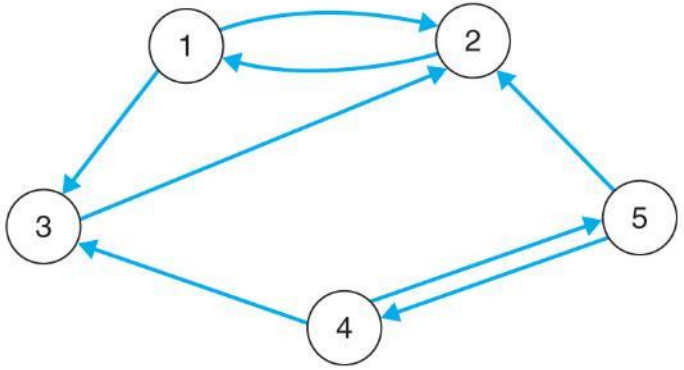


FIGURE 8.1B
The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

Graph Terminology

- A graph is an ordered pair G=(V,E) with a set of vertices or nodes and the edges that connect them
- A subgraph of a graph has a subset of the vertices and edges
- The edges indicate how we can move through the graph

Graph Terminology

- A path is a subset of E that is a series of edges between two nodes
- A graph is connected if there is at least one path between every pair of nodes
- The length of a path in a graph is the number of edges in the path

Graph Terminology

- A complete graph is one that has an edge between every pair of nodes

- A weighted graph is one where each edge has a cost for traveling between the nodes

Graph Terminology

- A cycle is a path that begins and ends at the same node
- An acyclic graph is one that has no cycles
- An acyclic, connected graph is also called an unrooted tree

Data Structures for Graphs an Adjacency Matrix

- A two-dimensional matrix or array that has one row and one column for each node in the graph
- For each edge of the graph (V_i, V_j), the location of the matrix at row *i* and column *j* is 1
- All other locations are 0

Data Structures for Graphs: Adjacency Matrix

There are two main data structures used to represent graphs. We assume the graph $G = (V, E)$ contains n vertices and m edges.

We can represent G using an $n \times n$ matrix M, where element $M[i, j]$ is, say, 1, if (i, j) is an edge of G, and 0 if it isn't. It may use excessive space for graphs with many vertices and relatively few edges, however.

Can we save space if (1) the graph is undirected? (2) if the graph is sparse?

Data Structures for Graphs
An Adjacency Matrix

- For an undirected graph, the matrix will be symmetric along the diagonal
- For a weighted graph, the adjacency matrix would have the weight for edges in the graph, zeros along the diagonal, and infinity (∞) every place else

Adjacency Matrix Example 1

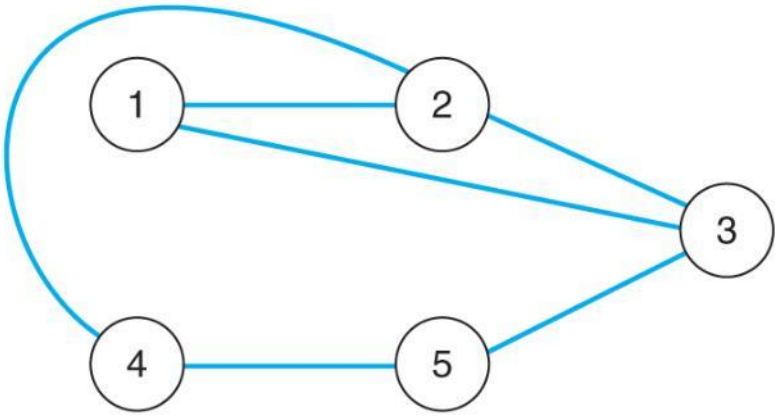


FIGURE 8.1A

The graph G = ({1, 2, 3, 4, 5}, {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}})

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

FIGURE 8.2A

The adjacency matrix for the graph in Fig. 8.1(a)

Adjacency Matrix Example 2

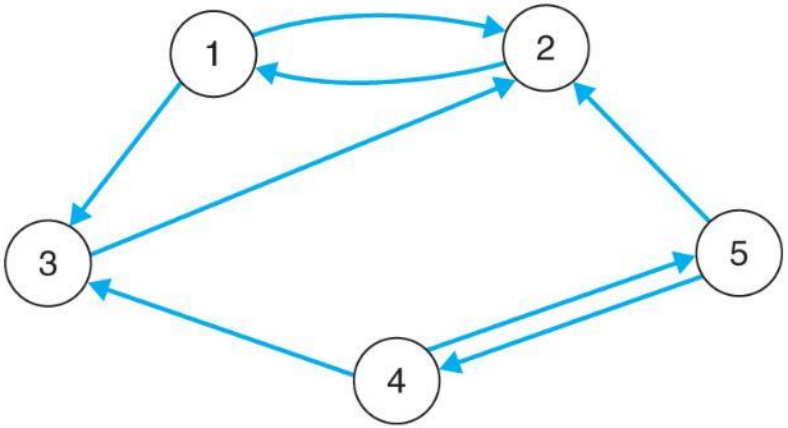


FIGURE 8.1B

The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

Adjacency Matrix Example 2

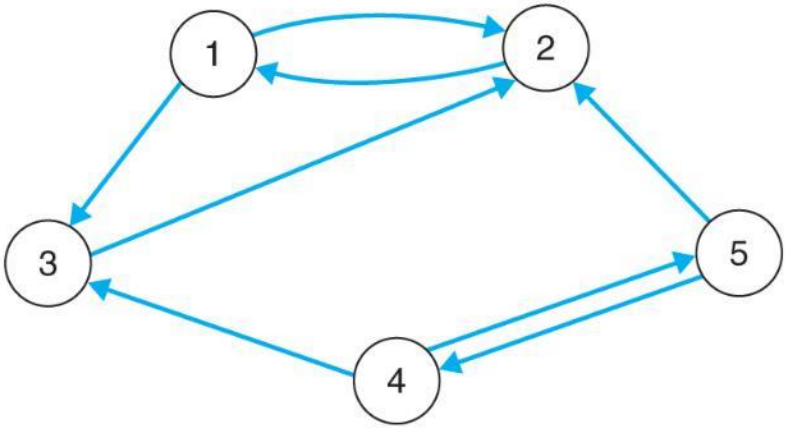


FIGURE 8.1B

The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	0	0
3	0	1	0	0	0
4	0	0	1	0	1
5	0	1	0	1	0

FIGURE 8.2B

The adjacency matrix for the digraph in Fig. 8.1(b)

Data Structures for Graphs: Adjacency Matrix

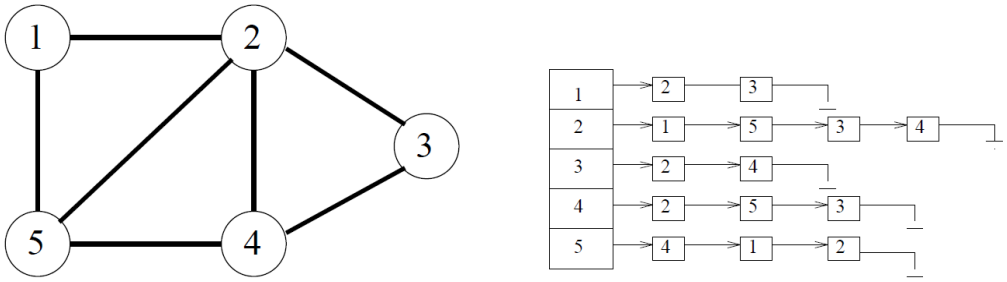
There are two main data structures used to represent graphs. We assume the graph $G = (V, E)$ contains n vertices and m edges.

We can represent G using an $n \times n$ matrix M, where element $M[i, j]$ is, say, 1, if (i, j) is an edge of G, and 0 if it isn't. It may use excessive space for graphs with many vertices and relatively few edges, however.

Can we save space if (1) the graph is undirected? (2) if the graph is sparse?

Adjacency Lists

An *adjacency list* consists of a $N \times 1$ array of pointers, where the ith element points to a linked list of the edges incident on vertex i.



To test if edge (i, j) is in the graph, we search the ith list for j, which takes $O(d_i)$, where d_i is the degree of the ith vertex. Note that d_i can be much less than n when the graph is sparse. If necessary, the two *copies* of each edge can be linked by a pointer to facilitate deletions.

Data Structures for Graphs
An Adjacency List

- A list of pointers, one for each node of the graph
- These pointers are the start of a linked list of nodes that can be reached by one edge of the graph
- For a weighted graph, this list would also include the weight for each edge

Adjacency List Example 1

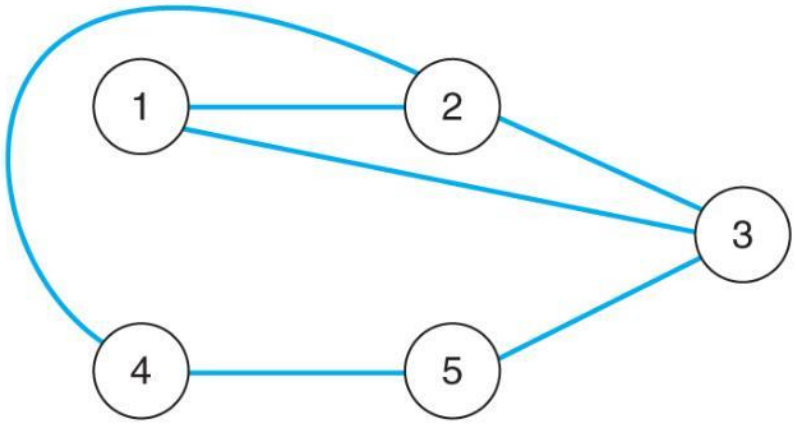


FIGURE 8.1A

The graph G = ({1, 2, 3, 4, 5}, {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}})

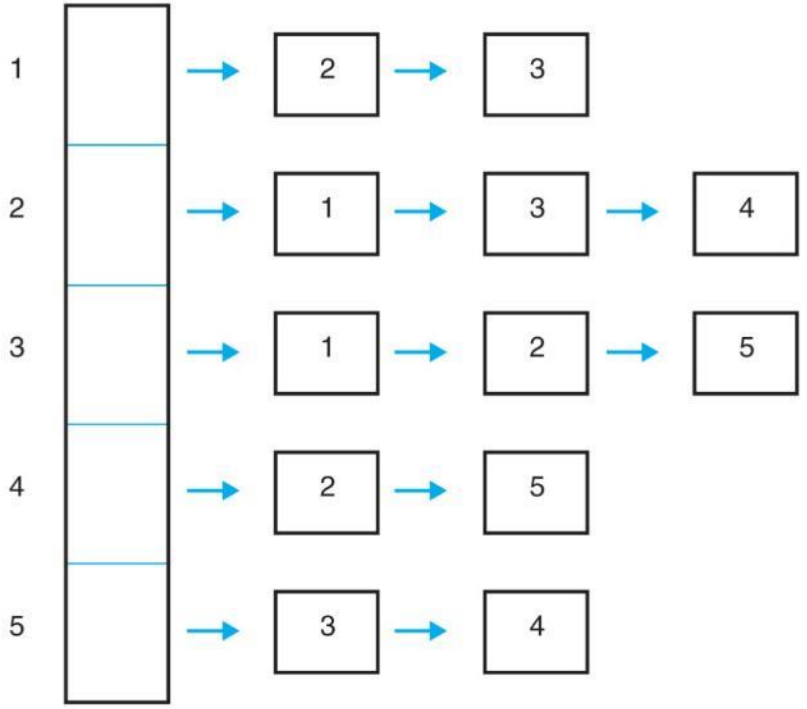


FIGURE 8.3A

The adjacency list for the graph in Fig. 8.1(a)

Adjacency List Example 2

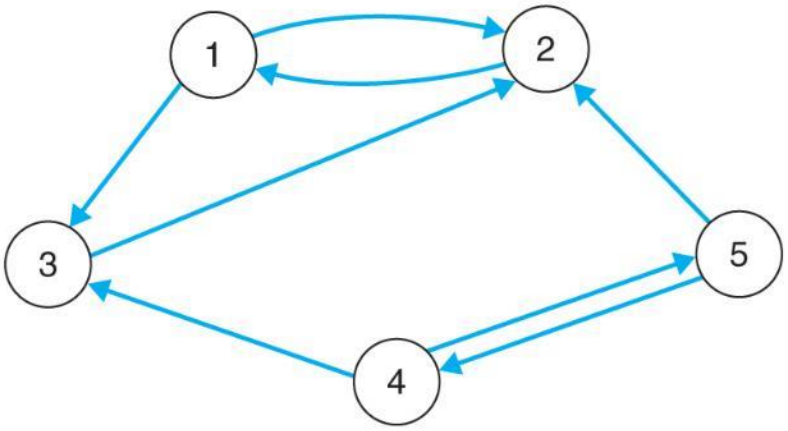


FIGURE 8.1B

The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

Adjacency List Example 2

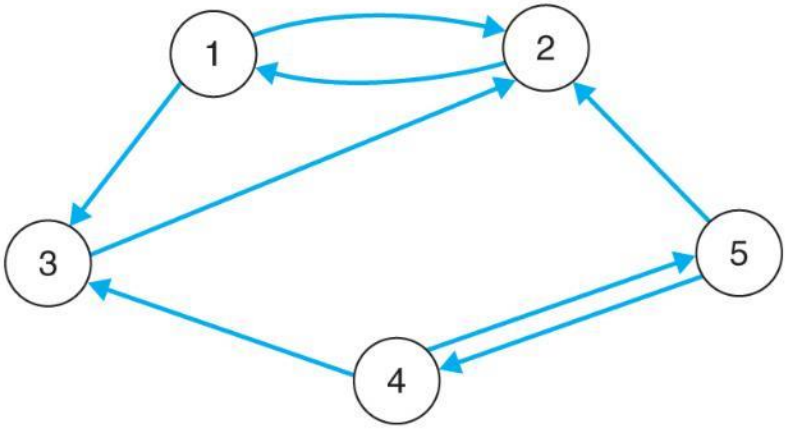


FIGURE 8.1B

The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

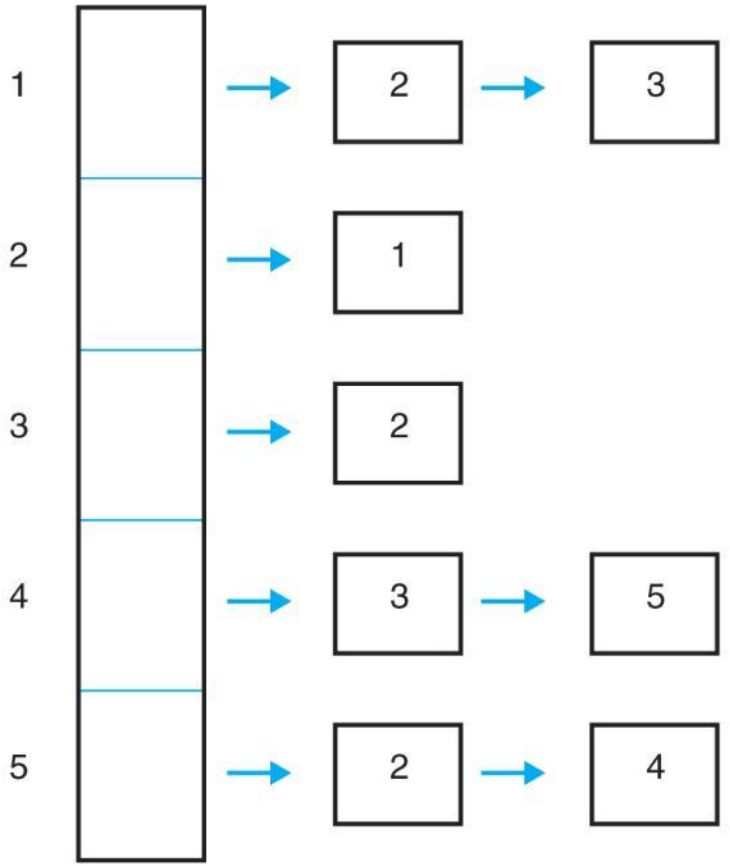


FIGURE 8.3B

The adjacency list for the graph in Fig. 8.1(b)

Tradeoffs Between Adjacency Lists and Adjacency Matrices

Comparison	Winner
Faster to test if (x, y) exists?	matrices
Faster to find vertex degree?	lists
Less memory on small graphs?	lists $(m + n)$ vs. (n^2)
Less memory on big graphs?	matrices (small win)
Edge insertion or deletion?	matrices $O(1)$
Faster to traverse the graph?	lists $m + n$ vs. n^2
Better for most problems?	lists

Both representations are very useful and have different properties, although adjacency lists are probably better for most problems.

Adjancency List Representation

```
#define MAXV 100

typedef struct {
        int y;
        int weight;
        struct edgenode *next;
} edgenode;
```

Edge Representation

```
typedef struct {
        edgenode *edges[MAXV+1];
        int degree[MAXV+1];
        int nvertices;
        int nedges;
        bool directed;
} graph;
```

The degree field counts the number of meaningful entries for the given vertex. An undirected edge (x, y) appears twice in any adjacency-based graph structure, once as y in x's list, and once as x in y's list.

Initializing a Graph

```
initialize_graph(graph *g, bool directed)
{
        int i;

        g -> nvertices = 0;
        g -> nedges = 0;
        g -> directed = directed;

        for (i=1; i<=MAXV; i++) g-> degree[i] = 0;
        for (i=1; i<=MAXV; i++) g-> edges[i] = NULL;
}
```

Reading a Graph

A typical graph format consists of an initial line featuring the number of vertices and edges in the graph, followed by a listing of the edges at one vertex pair per line.

```
read_graph(graph *g, bool directed)
{       int i;
        int m;
        int x, y;

        initialize_graph(g, directed);

        scanf("%d %d",&(g- >nvertices),&m);

        for (i=1; i<=m; i++) {
                scanf("%d %d",&x,&y);
                insert_edge(g,x,y,directed);
        }
}
```

Inserting an Edge

```
insert_edge(graph *g, int x, int y, bool directed)
{
        edgenode *p;

        p = malloc(sizeof(edgenode));

        p->weight = NULL;
        p->y = y;
        p->next = g->edges[x];

        g->edges[x] = p;

        g->degree[x] ++;

        if (directed == FALSE)
                insert_edge(g,y,x,TRUE);
        else
                g->nedges ++;
}
```

Graph Traversals

- We want to travel to every node in the graph
- Traversals guarantee that we will get to each node exactly once
- This can be used if we want to search for information held in the nodes or if we want to distribute information to each node

Depth-First Traversal

- We follow a path through the graph until we reach a dead end
- We then back up until we reach a node with an edge to an unvisited node
- We take this edge and again follow it until we reach a dead end
- This process continues until we back up to the starting node and it has no edges to unvisited nodes

Depth-First Traversal Example

- Consider the following graph:

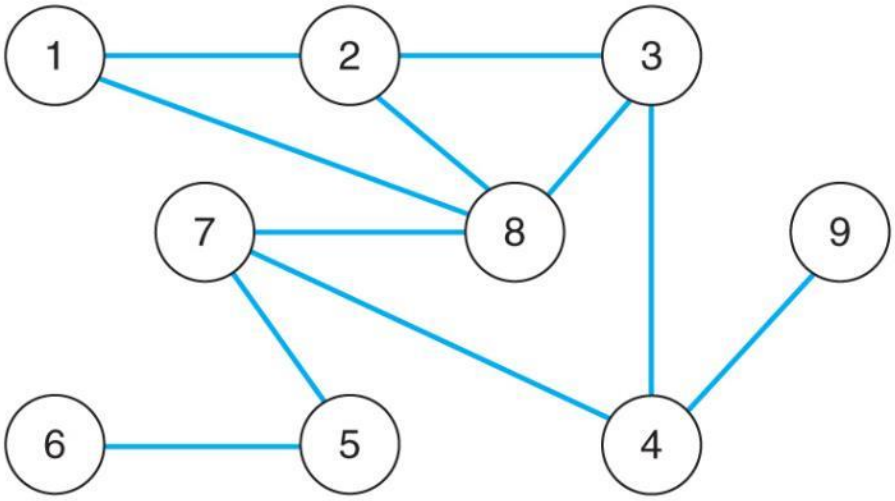


FIGURE 8.4
A graph

- The order of the depth-first traversal of this graph starting at node 1 would be: 1, 2, 3, 4, 7, 5, 6, 8, 9

Depth-First Traversal Algorithm

```
Visit( v )
Mark( v )
for every edge vw in G do
   if w is not marked then
      DepthFirstTraversal(G, w)
   end if
end for
```